# Partitioned-queue storage and lifetime contract

## 1 Authority and exactness boundary

This is the implementation companion to `core_math.tex`. Scheduling, layout, streaming, and recomputation policy may change; the sampled rollout law, completion estimators, $\lambda$-returns, posterior-weighted surrogates, clipping, entropy, reference KL, and distributed decision law may not. The algorithms in this directory must deliver the same mathematical arguments to every production-v1 loss. V1 implements the clipped PPO policy surrogate, value regression with optional value clipping, entropy, and legal-action KL to a frozen reference. It deliberately defers a separately stored $Q$ target/loss, RNaD/NeuRD, $W/S$ auxiliaries, and adaptive branching intensity. The corresponding definitions remain research material in `core_math.tex`, not latent allocation requirements.

Each partition has an exact loss domain $\mathcal{X}_m$, a natural model-row domain $\mathcal{Y}_m$, and a fixed architecture map $\varphi_m : \mathcal{X}_m \to \mathcal{Y}_m$. Model inputs and outputs are stored directly on $\mathcal{Y}_m$. Whenever a loss is summed over an exact private state $x \in \mathcal{X}_m$, it reads the corresponding model row $\varphi_m(x)$:

$$V^\theta(x, \sigma) = V^\theta_m[\varphi_m(x), \sigma], \qquad \log \pi^\theta(a \mid x, \sigma) = \log \pi^\theta_m[a, \varphi_m(x), \sigma].$$

Usually $\mathcal{X}_m = \mathcal{Y}_m$ and $\varphi_m$ is the identity. The stored arrays are simply the model's own arrays on $\mathcal{Y}_m$. Posterior, return, and advantage arrays remain on $\mathcal{X}_m$ because aggregating them is generally not loss-preserving under nonlinear PPO and value clipping.

All decisions routed to one partition must use the same canonical ordered exact-state domain $\mathcal{X}_m$. The meaning and row order of $x$ may not depend on a particular decision, board, owner, or physical batch. Impossible states have zero posterior mass. A game must split a partition if it cannot supply this invariant; core never infers or repairs a decision-local domain.

## 2 Game-neutral partition interface

The core sees only a finite collection of immutable `PartitionSpec` objects. A specification contains

| Member | Contract |
|---|---|
| $m$, diagnostic name | Stable integer identity; the name is never parsed by core. |
| $\kappa(\sigma)$ | Pure public-state router for player nodes. |
| $\preceq$ | Optional preorder used only to choose among otherwise eligible queues. |
| $X_m = \lvert\mathcal{X}_m\rvert$ | Exact private-state width used by targets and losses. |
| $Y_m = \lvert\mathcal{Y}_m\rvert$, $\varphi_m$ | Natural model-row width and fixed exact-state-to-model-row lookup. |
| input/model layout | Natural partition input and curried policy/value forward on $\mathcal{Y}_m$. |
| byte/work contract | Fixed worst-case retained bytes per row, transient bytes per physical pack, safe row limits, and legal batch quanta. |
| completion specification | Opaque immutable leaf fields, exact ordered reach domain, counter-based seed/key, declared output rows, and calibrated completion bytes/work. |

Core may route, queue, account, sample, and batch by $m$. It may not inspect a game stage, synthesize a subpartition, or reorder rows by an undeclared public label. The default scheduler is full-batch first; within the set of full queues it chooses a minimal preorder layer, then load and age. A forced partial drain also starts in the minimal nonempty layer. With an empty preorder all partitions are incomparable.

# 3 Poker application of the abstract interface

Poker supplies exactly

$$\mathcal{M} = \{\mathsf{pre}\} \sqcup \{\mathsf{post}_A : 2 \le A \le P\}, \qquad \mathsf{pre} \prec \mathsf{post}_A,$$

with the $\mathsf{post}_A$ mutually incomparable. The public board and betting history remain opaque input fields. Flop, turn, and river with the same $A$ are one partition, use the same fixed dense hand width, and are not sorted, split, packed, or assigned different physical costs by street. This contract contains no optimization based on structural zeros after a board card is revealed.

Let

$$K = R^2, \qquad H = \binom{4R}{2}, \qquad b_f = 4, \qquad b_i = 4.$$

The model-input byte declarations are

$$I_{\mathsf{pre}} = I^{\text{public}}_{\mathsf{pre}} + b_f PK,$$
$$I_{\mathsf{post}_A} = I^{\text{public}}_{\mathsf{post}_A} + b_f AH + b_i \mathbb{1}_{A<P}.$$

Only live players contribute exact-hand postflop ranges. The last term is an immutable reference to one folded-player summary stored at its dependency anchor; it is not an inlined summary. Poker creates a new anchor exactly when the summary dependencies change: on a fold event or a public-street transition, following `MHA_model.md` and `folder_impact.md`. Every descendant decision references that immutable anchor until the next such event. A street transition therefore refreshes poker-owned input data; it does not create a street partition or change the fixed $\mathsf{post}_A$ layout. In particular, $\mathsf{post}_P$ has no summary reference and never materializes a neutral summary.

RB leaf completion is the only game-specific terminal boundary. For poker, `CompletionSpec` is the binding contract documented in the read-only `/home/pablob/poker-gpu-evaluator/docs/api_pbg.md`: the leaf owns the required contiguous board, range-weight, dealt/contender, pot-stratum, baseline, amount, and logical-seed fields in their declared dtypes. The core calls the sibling evaluator and extracts only posterior-mass and value rows needed by the target algorithms; it must not reimplement equity, posterior, or Monte Carlo completion locally. Completion inputs, batching, outputs, scratch, and backend workspace are part of the hard byte contract.

The preflop architecture is intrinsically defined on the ordinary quotient $c : \{1, \dots, H\} \to \{1, \dots, K\}$, which is $1326 \to 169$ for the full $R = 13$ deck. Thus

$$Y_{\mathsf{pre}} = K, \quad \varphi_{\mathsf{pre}} = c, \qquad Y_{\mathsf{post}_A} = H, \quad \varphi_{\mathsf{post}_A} = \mathrm{id}\,.$$

Preflop model inputs and every materialized or retained dense current, old, and reference policy/value output are exactly $K$-dimensional; no staging, storage, or cache may expand them to $H$. Postflop model inputs and every materialized or retained dense current, old, and reference policy/value output are exactly $H$-dimensional, including zero rows for board-blocked hands. Normal mode may retain only its selected scalar loss slices; that is a strict projection, not an $H$-wide representation. Crucially,

$$X_{\mathsf{pre}} = X_{\mathsf{post}_A} = H$$

for RB posteriors, returns, advantages, and losses. At preflop an exact-hand loss term for $h$ simply uses model row $c(h)$; no duplicate $H$-dimensional model output is retained.

# 4 Stable row domains and owning objects

Let $D_m$ be player decisions in partition $m$, $D = \sum_m D_m$, $L$ stopped leaves, and $G$ sequential trajectories. Sequential modes have one implicit sampled edge per decision and allocate no separate action table. In branched RB, $A_m$ is the number of sampled outgoing player edges in partition $m$ and $A = \sum_m A_m$.

| Object | Row domain | Ownership contract |
|---|---|---|
| `OccurrenceArena` | live rollout occurrences | Mutable public-state handle, active reach/profile, logical RNG address, child reservation, status, stable table address, and sequential root/decision ordinal when applicable; rollout and seal source only. |
| `SequentialDecisionCSR` | $D$ decisions in $G$ paths | Decision-only chronological paths plus one leaf row per trajectory; chance nodes are absent. |
| `BranchedTreeSpan` | $N$ nodes | One rooted tree; every node owns a contiguous child span. The root uses the ordinary branching rule. |
| `DecisionTable[`$m$`]` | $D_m$ player nodes | Owner, implicit-edge data in sequential modes, natural old-value rows, decision targets, and posterior reference as required by mode. |
| `ActionTable[`$m$`]` | $A_m$ player edges, branched only | CSR pointer, event, natural selected old-log-probability rows, child posterior, and edge targets; no padded action axis. |
| `PartitionInputTable[`$m$`]` | $D_m$, aligned | Immutable natural model input; no all-node public-state reference. |
| `LeafTable` | $L$ stopped paths | Normal terminal reward, or RB generated reach plus immutable completion input and RNG key. |
| `SequentialPosteriorPool` | required $(i, p)$ pairs | Each sequential completion posterior is retained once and referenced by all decisions of that player on trajectory $i$. |
| `TrainingStorage` | one local iteration | Only immutable model inputs, natural old model rows, exact targets/posteriors, and sampling metadata. |

A decision receives $(m, d)$ when its player occurrence is created. A branched player edge receives $(m, a)$ when its logical children are reserved, before any physical model pack. Reservation order is stable, and the $u$th action row is aligned with the $u$th contiguous child. Physical packing may split this work but cannot alter an address, draw, or topology.

Rollout arenas and growing tables are chunked and append-only. Their row identities are logical integers, never pointers into reallocatable tensors. After rollout, `SealRolloutStorage` in `family_scheduled_rollout.tex` allocates exact contiguous destination columns, copies stable spans once, validates the topology, and only then releases source chunks. This is an explicit phase: admission charges source capacity, destination capacity, and bounded seal scratch simultaneously. No claim of zero-copy ownership transfer is part of the design.

# 5 Normative retained fields

All reaches, posterior masses, moments, returns, advantages, values, and loss log probabilities are fp32. A model or reference forward may retain its ordinary inference dtype when that is already the defined arithmetic of the forward. Identifiers are int32; logical RNG keys and mass totals are int64/fp64.

## Normal sequential

| Owner | Field | Shape |
|---|---|---|
| `PartitionInputTable[`$m$`]` | natural model input | $[D_m, I_m]$ bytes |
| `DecisionTable[`$m$`]` | owner, event, realized type | $3[D_m]$ int32 |
| | old value, selected old log probability | $2[D_m]$ fp32 |
| | `lambdaReturn` | $[D_m]$ fp32 |

$\widehat{V}$ = `lambdaReturn` and $\widehat{A}$ = `lambdaReturn` $- V^{\theta_k}$ are loss-time expressions, not arrays.

### Sequential RB

| Owner | Field | Shape |
|---|---|---|
| `PartitionInputTable[`$m$`]` | natural model input | $[D_m, I_m]$ bytes |
| `DecisionTable[`$m$`]` | owner, event, posterior reference | $3[D_m]$ int32 |
| | old value and selected log probability | $2[D_m, Y_m]$ fp32 |
| | `lambdaReturn` | $[D_m, X_m]$ fp32 |
| `SequentialPosteriorPool` | $\widehat{\mathbb{P}}(-_p \mid \mathsf{T}_i^{\mathsf{pre-c}})$ | one $[X_{m(i,p)}]$ fp32 row per required $(i,p)$ |

The parent and unique-child posterior of every decision are the same referenced completion row. V1 stores no separate $Q$ target and exposes no $Q$ loss.

### Branched RB

| Owner | Field | Shape |
|---|---|---|
| `PartitionInputTable[`$m$`]` | natural model input | $[D_m, I_m]$ bytes |
| `DecisionTable[`$m$`]` | owner; old value | $[D_m]$ int32; $[D_m, Y_m]$ fp32 |
| | `valueHat` | $[D_m, X_m]$ fp32 |
| `ActionTable[`$m$`]` | action pointer; event | $[D_m + 1]$, $[A_m]$ int32 |
| | selected old log probability | $[A_m, Y_m]$ fp32 |
| | child posterior, advantage | $2[A_m, X_m]$ fp32 |

The node posterior is exactly the mean of its sampled child posterior rows and is derived in the loss gather. It is never retained separately. Fixed- intensity PPO v1 stores only the value and advantage arguments above.

Every mode stores fp64 $\log Z_m$ for nonempty partitions. Nonuniform branched strata store one alias threshold (fp64) and alias index (int32) per decision; uniform sequential strata use an allocation-free implicit sampler. Branched $\log \omega_d = -\sum_{s \prec d} \log N(s)$ exists only through sampler construction and is then released.

## 6 Compact topology

`SequentialDecisionCSR` contains only

$$\texttt{trajectoryPtr}[G+1], \quad \texttt{decisionPartition}[D], \quad \texttt{decisionRow}[D], \quad \texttt{leafRow}[G].$$

Decision owner/event/type live in the partition row. Chance nodes do not advance a player clock and need no backward row. No sampled private profile survives rollout: normal leaves already own exact terminal rewards.

`BranchedTreeSpan` contains int32

$$\texttt{parent}, \ \texttt{childBegin}, \ \texttt{degree}, \ \texttt{decisionPartition}, \ \texttt{decisionRow}, \ \texttt{leafRow} \quad [N].$$

Sentinel $-1$ marks an inapplicable field. Children of every node are contiguous, so there is no `childNode`$[E]$, duplicated child event, or stored $\nu[E]$: $\nu = 1/\texttt{degree}$ is implicit. At a player node the action span has the same order as the child span.

Thus the finalized topology sizes are exactly

$$T_{\text{seq}} = b_i(2D + 2G + 1), \qquad T_{\text{branch}} = 6b_iN.$$

## 7 Write times and lifetime schedule

The generated reach is written once at its last necessary destination: natural decision input at player-node creation, exact RB leaf reach at stopped leaf creation. Chance-only reaches and mutable public states live only until their descendants are materialized. Once a processed parent has committed all children, its mutable public state and active reach/profile are released; only compact topology fields remain. A stopped leaf releases the same payload immediately after its immutable reward or completion row is written. No target builder replays a path to reconstruct reach, and no $[N, P, X]$ reach memo exists.

| Phase | Resident | Released before next phase |
|---|---|---|
| Rollout | read-only $\theta_k$; mutable public states; occurrence arena; queues; active reaches/profiles; growing chunked input/old/action/leaf tables; one bounded physical pack | each physical output immediately; then queues, active rows, and mutable states |
| Seal | rollout source-chunk capacities; exact contiguous topology/input/old/action/leaf destinations; bounded copy/scan scratch | source chunks and seal scratch after synchronized validation |
| Targets | compact topology; partition input/old tables; leaf table; final target arrays; one completion microbatch; sequential carries or prebounded branched slot arena | every extra completion result immediately; reduced slots immediately; then leaf table, topology, old log weights, validity flags, and target scratch |
| Optimization | immutable training storage; optional selected reference caches; model/reference/optimizer; flat gradient SUM buffer; one reusable bounded physical-subspan arena | each subspan's graph, gathers, reductions, and uncached reference output immediately after backward; then reference cache and training storage after the final optimizer step |

Synchronous training needs no frozen copy of $\theta_k$: parameters remain read-only until all GPUs have fixed targets and enter the first optimizer step. A future overlapped rollout/optimization pipeline would require an explicit snapshot and a different lifetime proof; it is not part of these algorithms. "Release" means the last owning reference is gone and the storage can be reused. Allocator reserved bytes and backend workspaces are measured separately; a Python name deletion alone is not a memory event.

# 8 Old implementation pattern versus new invariant

| Old pattern | New invariant | Primary effect |
|---|---|---|
| Full-capacity or global-$H$ decision reach buffers | Append the natural partition input at decision creation | Removes unused capacity and the unnatural preflop $K \to H$ expansion. |
| Root cache followed by recursive `recompute` into an all-node reach memo | A generated decision or leaf reach is written once to its last consumer | Removes replay compute, Python traversal, and $[N, P, X]$ memo. |
| Global maximum action axis and padded target tensors | Branched sampled edges use CSR; sequential sampled edge is implicit | Storage and gathers scale with sampled edges, not $DE_{\max}$. |
| Whole logical-batch outputs retained while children are created | Reserve all logical addresses first; forward, materialize, and release one physical subspan at a time | Bounds inference staging without changing topology or RNG. |
| All-node sequential topology including chance nodes | Decision-only trajectory CSR | Reverse work scales with decisions; player clocks remain exact. |
| Separate sequential value, advantage, and return arrays plus repeated posterior rows | Retain one `lambdaReturn`, derive $V/A$, and reference one completion posterior per trajectory/player | Removes exact algebraic duplicates, not estimator information. |
| Stored branched parent posterior and child posterior | Retain child posterior; derive the parent mean in the node loss | Saves $b_f \sum_m D_m X_m$. |
| Mandatory frozen rollout-model clone | Keep $\theta_k$ read-only until synchronous optimization starts | Removes one model snapshot in the specified lifecycle. |
| Reference forward repeated for every sampled learner row | Cache a selected partition's natural reference outputs only when saved time justifies bytes | Trades memory for compute explicitly rather than globally. |
| Gather a full statistical partition minibatch before physical splitting | Draw only decision indices; plan, gather, backward, and release one feasible physical subspan at a time | Makes the training transient depend on the largest live subspan, not $B_m$. |

| Old pattern | New invariant | Primary effect |
|---|---|---|
| Neutral folded-player summaries materialized for no-fold states | Summary reference exists only when $A < P$ and its data is anchored once | Avoids large repeated neutral tensors. |
| Copy-flatten gradients before reduction | Parameter gradients are views into one preallocated fp32 SUM buffer | Removes a full-gradient copy and allocator churn per step. |

The design does not prohibit all recomputation. Cheap loss-time expressions ($\widehat{V}$ = `lambdaReturn`, $\widehat{A}$ = `lambdaReturn` $- V^{\theta_k}$, and a branched parent-posterior mean) are deliberately derived, and a reference forward may be recomputed when its cache would reduce safe decision capacity. What is forbidden is replaying discarded rollout state or changing an estimator to save bytes.

# 9 Byte formulas

Let $Y_m = |\mathcal{Y}_m|$ be the partition's natural model-row width, and let

$$M_{\text{post}}^{\text{seq}} = b_f \sum_{(i,p)\in\mathcal{U}} X_{m(i,p)}$$

be the sequential posterior pool for the set $\mathcal{U}$ of trajectory/player pairs that own at least one sampled decision. Define

$$\begin{aligned}
C_{\text{normal}} &= 3b_iD,\\
C_{\text{seqRB}} &= 3b_iD,\\
C_{\text{branch}} &= b_iD + b_i\sum_m (D_m + 1) + b_iA,\\
M_{\text{sample}} &= b_{64}|\mathcal{M}_+| + \sum_{m\in\mathcal{M}_{\text{nonuniform}}} D_m(b_{64} + b_i).
\end{aligned}$$

Excluding optional reference caches and game-owned summary anchors, the compact training stores are

$$\begin{aligned}
M_{\text{normal}} &= \sum_m D_m(I_m + 3b_f) + C_{\text{normal}} + M_{\text{sample}},\\
M_{\text{seqRB}} &= \sum_m D_m(I_m + 2b_fY_m + b_fX_m) + M_{\text{post}}^{\text{seq}} + C_{\text{seqRB}} + M_{\text{sample}},\\
M_{\text{branchRB}} &= \sum_m D_m(I_m + b_fY_m + b_fX_m)\\
&\quad + \sum_m A_m(b_fY_m + 2b_fX_m) + C_{\text{branch}} + M_{\text{sample}}.
\end{aligned}$$

Let $B_{\text{reach}} = b_f \sum_{p\in\mathcal{P}} |\mathcal{X}_p|$ and let $C_L$ be completion-public-input bytes per RB leaf. Leaf stores are

$$M_L^{\text{normal}} = L(b_i + b_fP), \qquad M_L^{\text{RB}} = L(b_i + b_{64} + C_L + B_{\text{reach}}).$$

The first int is terminal kind; topology owns the leaf address.

Hard accounting uses allocated capacities, not logical payload alone. For every chunked column $e$, let $\text{cap}(e)$ be its allocated row capacity and $b(e)$ its complete schema bytes per row, including alignment. Define

$$M_{\text{chunks}} = \sum_e \text{cap}(e)b(e) + M_{\text{chunk-header}} + M_{\text{allocator-slack}}.$$

The immutable schema, rather than a handwritten field count, generates this sum and is shared by allocation, prediction, and ledger validation. It must include separate logical RNG address/key components, log weight, status, root/ordinal, table addresses, and every mode-specific field. Appending one row first transactionally admits the next chunk's full capacity.

Let $S$ be worst-case mutable public-state bytes per resident state, $F_{\text{state}}$ the maximum number of simultaneously resident mutable public states (queued, evaluating, or being chance-closed), $Q_{\text{cap}}$ allocated queue-handle capacity, $C$ maximum live sequential trajectories, $M_{\text{summary}}$ the game-owned immutable summary-anchor store, and

$$\begin{aligned}
M_{\text{state}} &\le F_{\text{state}}S,\\
M_{\text{queues}} &= b_i(Q_{\text{cap}} + 2|\mathcal{M}|),\\
M_{\text{active}}(C) &= C(B_{\text{reach}} + b_iP + 2b_i).
\end{aligned}$$

Thus state capacity follows the live-frontier high-water mark, not total generated occurrences $N$; completed occurrences retain only their bounded arena/topology metadata. Let $R_{\mathsf{m}}$ be the mode's exact sealed input, metadata, and old-output fields before targets; let $M_{\text{grad}}$ be the one flat fp32 gradient SUM buffer, $M_{\text{collective}}$ its fixed reduction workspace, and $M_{\text{index}} = b_i \max_m B_m^{\text{cap}}$ one reusable index-only draw buffer sized by the statistical time-budget caps; and let $W_{\text{infer}}$, $W_{\text{seal}}$, $W_{\text{completion}}$, and $W_{\text{backward}}$ be calibrated hard transient bounds.

For optimization, $W_{\text{train},m}(J;\mathcal{C})$ is the allocator-reserved high-water increment for one physical decision-index subspan $J$ in partition $m$ with reference-cache set $\mathcal{C}$. It includes only that subspan's input/target/cache gathers, branched action rows, uncached reference output, current-model activations, exact-loss reductions, AMP temporaries, and backend workspace. Define

$$\begin{aligned} M_{\text{opt}}^{\text{base}} &= M_{\text{static}} + M_{\mathsf{m}} + M_{\text{summary}} + M_{\text{refcache}} \\ &\quad + M_{\text{grad}} + M_{\text{collective}} + M_{\text{index}}, \\ W_{\text{train}} &= \max_{m,J \text{ executed}} W_{\text{train},m}(J;\mathcal{C}), \\ W_{\text{refbuild}} &= \max_{m\in\mathcal{C},J \text{ built}} W_{\text{refbuild},m}(J;\mathcal{C}), \end{aligned}$$

where $W_{\text{refbuild}} = 0$ when $\mathcal{C} = \varnothing$. Because every subspan is backwarded and released before its successor, neither statistical counts $B_m$ nor different partitions are added in $W_{\text{train}}$. Activation checkpointing is not part of the default bound; it is admitted only as a separately measured physical-shape variant when its recomputation improves end-to-end spots/second, or when no uncheckpointed singleton fits. Then

$$\begin{aligned} M_{\text{rollout}} &= M_{\text{static}} + M_{\text{state}} + M_{\text{chunks}}^{\text{high}} + M_{\text{queues}} + M_{\text{active}}(C) + M_{\text{summary}} + W_{\text{infer}}, \\ M_{\text{seal}} &= M_{\text{static}} + M_{\text{chunks}}^{\text{final}} + T_{\mathsf{m}} + R_{\mathsf{m}} + M_L + M_{\text{summary}} + W_{\text{seal}}, \\ M_{\text{targets}} &= M_{\text{static}} + T_{\mathsf{m}} + M_{\mathsf{m}} + M_L + M_{\text{summary}} + W_{\text{completion}} + W_{\text{backward}}, \\ M_{\text{optimization}} &= M_{\text{opt}}^{\text{base}} + \max\{W_{\text{train}}, W_{\text{refbuild}}\}, \\ M_{\text{peak}} &= \max\{M_{\text{rollout}}, M_{\text{seal}}, M_{\text{targets}}, M_{\text{optimization}}\}. \end{aligned}$$

No mutable all-node public-state store appears in target or optimization phases. Objects with disjoint lifetimes are not added. Admission uses allocator-reserved/NVML bytes, not only live tensor bytes. With usable device capacity $M_{\text{device}}$ and intentionally unused guard $M_{\text{guard}}$, every cache choice and physical plan must satisfy both

$$M_{\text{opt}}^{\text{base}} + \max\{W_{\text{train},m}(J;\mathcal{C}), W_{\text{refbuild},m'}(J';\mathcal{C})\} \le M_{\text{hard}} := M_{\text{device}} - M_{\text{guard}} \quad \text{for every executed training or cache-build su}$$

On a 2080 Ti constrained to 10.5 GiB usable VRAM, take initially $M_{\text{guard}} = 0.75$ GiB and hence $M_{\text{hard}} = 9.75$ GiB. The guard covers measurement error, allocator fragmentation, and unmodeled backend-workspace variation. The ceiling may be raised only after each phase's reserved-byte high-water mark is stable across representative iterations; an OOM is not a calibration method.

# 10 Memory-only sequential root-width control

Normal and sequential-RB runs choose a common per-GPU root width $G_k$ with Algorithms `ChooseSequentialRootWidth` and `ObserveSequentialMemory` in `sequential_memory_control.tex`. There is intentionally no player-node or throughput target in v1. The controller solves the simpler root-only analogue of choosing $\beta(\varnothing)$ and then fixing $\beta(\sigma) = 1$ below the root: it selects the largest lagged width whose upper prediction fits every physical phase, then applies conservative growth hysteresis.

The predictor is schema-derived and phase-specific. It combines exact fixed bytes with one-sided per-root estimates for realized decisions, occurrences, leaves, chunk capacity/slack, summary anchors, seal overlap, posterior rows, and calibrated workspace. It uses completed prior iterations only; $G_k$ and all root keys are frozen before any current draw. Ranks use an all-reduced minimum safe width so synchronous work and collective order agree. Unsafe evidence decreases width immediately, whereas an increase is capped and requires a run of successful covered iterations. State and calibration are checkpointed.

This is a capacity controller, not a geometry rejection rule. Neither it nor `SafeAdmit` may truncate, discard, reroll, or replace already selected roots. `SafeAdmit` only limits concurrent live frontier work. If the declared transaction for a chunk, seal, completion, or singleton learner subspan cannot fit, the run checkpoints and stops cleanly; it does not alter the conditional sampling law. Optional reference caches are excluded from root feasibility and may be admitted later only when both cache construction and training fit. Consequently the empty-cache path is always the safety fallback.

At every phase boundary, terminal local failures participate in one declared common-order control agreement before any later collective. If any rank fails, all ranks release the current partial/sealed store and terminate from the last completed checkpoint. A rank must never skip a collective while healthy peers enter it, and no failed current iteration is retried.

# 11 HU 32 bb sequential-RB back-of-the-envelope

For the measured iteration in `runs/nlhe_hu_32bb_sequential_rb`,

$$P = 2,\ H = 1326,\ K = 169,\ D_{\mathsf{pre}} = 3889,\ D_{\mathsf{post}_2} = 3506,\ D = 7395,\ G = L = 2560,\ N = 11319.$$

The observed retained learner store is 274,805,595 bytes. Its old dense decision input is

$$DPHb_f = 78{,}446{,}160 \text{ bytes.}$$

Using the model's natural $K$-row preflop input and $H$-row postflop input gives

$$D_{\mathsf{pre}}PKb_f + D_{\mathsf{post}_2}PHb_f = 42{,}449{,}576 \text{ bytes.}$$

Natural-width old value and selected-log-probability rows together cost another 42,449,576 bytes. Exact-hand return targets cost

$$DHb_f = 39{,}223{,}080 \text{ bytes,}$$

and one posterior per completion/player costs

$$GPHb_f = 27{,}156{,}480 \text{ bytes.}$$

The new dense total is therefore 151,278,712 bytes. Reusing the measured 244,035-byte non-dense residual gives approximately 151,522,747 bytes (144.5 MiB), a 44.9% retained-store reduction. This estimate keeps every RB target and posterior at exact hand width $H$.

Generated decision-plus-leaf reach retention falls from

$$(D + L)PHb_f = 105{,}602{,}640$$

to $42{,}449{,}576 + 27{,}156{,}480 = 69{,}606{,}056$ bytes for this run. More importantly, removing root/all-node replay caches avoids a further 147,228,432 bytes plus recursive replay work. Materializing a neutral folded summary for $\mathsf{post}_2$ would waste 1,214,512,640 bytes; the $A = P = 2$ input contract makes that allocation exactly zero.

The measured model-row counts were 7,395 rollout rows, 22,208 learner rows, and 22,208 repeated reference rows. With two raise sizes, the poker reference head has ten native fp16 sufficient scalars per private model row, so caching all distinct retained inputs costs

$$2 \cdot 10(D_{\mathsf{pre}}K + D_{\mathsf{post}_2}H) = 106{,}123{,}940 \text{ bytes} \quad (101.2 \text{ MiB}).$$

It replaces 22,208 reference rows by 7,395, eliminating 14,813 of 51,811 total model rows (28.6%). New retained storage plus this cache is about 245.7 MiB, still below the old retained store. This makes full caching attractive for this measured shape, but the implementation must retain the per-partition knapsack choice: under a tighter phase peak, cache the partition with the largest measured reference time saved per native byte.

The same trace reported 647.4 of 701.0 seconds in optimization, so after removing replay and pathological summary storage, end-to-end spots/second is primarily a learner-throughput problem. The decision-count capacity must be chosen by the maximum of the four phase formulas, not by training-store bytes alone.

# 12 Implementation order and acceptance gates

The recommended order is:

1. immutable schemas, exact capacity ledgers, stable decision/leaf reach ownership, and complete removal of reach replay;
2. decision-only sequential topology, contiguous branched child spans, and logical reservation before bounded physical forwards, followed by explicitly budgeted exact sealing;
3. exact target deduplication while retaining old outputs at natural model width;

4. the memory-only sequential root-width controller and checkpoint state;
5. per-partition reference-cache selection, including build admission, and direct flat gradient views;
6. index-only statistical draws followed by plan–gather–backward–release physical subspans, calibrated separately by partition;
7. only then tune queue thresholds, statistical stratum counts, and the small warmed physical-shape sets from measured reserved bytes, kernel occupancy, and end-to-end spots/second.

Acceptance requires randomized small-PBG equality tests against direct `core_math.tex` formulas for all three modes, including $\lambda = 0, 1$, one/many branched children, zero posterior mass, and every production-v1 loss. It also requires per-phase allocated/reserved peaks, per-partition time/physical-byte curves, tail-subspan tests, and fixed-seed rollout, target, and optimization timings. Statistical counts $B_m$ remain arbitrary positive integers; only physical shapes are calibrated, and no universal multiple-of-32 rule is imposed. For one fixed sampled index vector, an unsplit reference calculation and every legal multi-subspan plan must produce the same $\alpha_m/B_m$-normalized objective and gradient within the declared arithmetic tolerance, including a one-row tail and cache-versus-recompute reference paths. Sealing tests must force multiple source chunks and verify exact destination extents, stable row/topology order, and a measured peak no larger than the declared source-plus-destination-plus-scratch transaction. Controller tests must cover cold start, immediate decrease, delayed/capped increase, multi-rank minimum, checkpoint resume, underestimated geometry, and clean pre-draw or singleton failure with no reroll. Poker tests must assert that all stored preflop model inputs/outputs/caches have width $K$ (169 for $R = 13$), all stored postflop model inputs/outputs/caches have width $H$ (1326 for $R = 13$), and RB posteriors/targets retain the canonical $H$ exact-hand order in both cases. No postflop-street specialization should be added without a separate controlled experiment showing a material end-to-end gain at equal sampled spots, identical estimators/losses, and no loss of safe decision capacity; the current design intentionally contains none.

# 13 Traceability to future game-neutral units

| Contract | Intended implementation unit |
|---|---|
| `PartitionSpec`, router/preorder/natural layouts | `pbg/core/partition.py` |
| queues, occurrence arena, stable reservations | `pbg/core/execution.py` |
| compact sequential/tree topology | `pbg/core/topology.py` |
| partition/input/action/leaf/training stores | `pbg/core/storage.py` |
| partitioned queue rollout | `pbg/core/rollout.py` |
| streaming exact target builders | `pbg/core/targets.py` |
| stratified distributed update | `pbg/core/training.py` |
| poker router, $H \to K$ quotient, summary anchors | `pbg/games/poker/` |

These are future game-neutral boundaries, not authorization to modify Python as part of this pseudocode change.